EffortGraph: Joint Model and Reasoning Effort Selection with Graph Neural Networks

Chenyue Li
Stanford University
chenyuel@stanford.edu

Yanzhen Shen
Stanford University
yanzhen4@stanford.edu

1 Introduction

Choosing how much computation to spend on a query requires deciding both which LLM to use and how much reasoning it should perform. In our proposal, we use a heterogeneous graph to share information among tasks, queries, models, and reasoning efforts. We ask whether this shared structure yields better quality–cost tradeoffs than fixed configurations and non-graph routing. So far, we have collected and scored responses from all nine configurations and trained the MLP baseline. We also tried changing the training-label distribution and fine-tuning the text encoder. We have not yet implemented the graph router.

We find that fine-tuning the encoder improves some results on the rebalanced training set, but does not consistently outperform the original MLP. Our code, data splits, predictions, and results are available at `https://github.com/yanzhen4/cs224w_effortgraph`.

2 Datasets and experimental setting

Benchmarks. We sample 800 questions across 15 dataset-specific task groups (Table 1), using MMLU-Pro [2], SuperGPQA [3], and LiveBench [4]. SuperGPQA contributes 20 easy, 20 middle, and 20 hard questions per field. LiveBench sampling covers subtasks and available native difficulty labels. This sample includes 127 questions that were later removed from LiveBench. We remove duplicate questions within each sampling pool.

Table 1: Sampled questions and task groups. Each fold holds out one fifth of each group.

Dataset	Task groups	Questions
MMLU-Pro	Math, physics, health, law, history	250
SuperGPQA	Physics, mathematics, chemistry, clinical medicine, biology	300
LiveBench	Mathematics, reasoning, coding, data analysis, instruction following	250

Configurations and grading. Each question is evaluated with `gpt-5-nano`, `gpt-5-mini`, and `gpt-5` (2025-08-07 snapshots), each at low, medium, and high reasoning effort. This gives 7,200 recorded responses. We use zero-shot prompts and a 64,000-token output limit including reasoning. Multiple-choice responses are scored by extracted-answer exact match; LiveBench uses task-specific grading, including code execution and an equivalence-judge fallback for AMPS mathematics. Three incomplete responses receive zero credit. Quality is the benchmark score, including partial credit where supported. Cost is calculated from recorded input, cached-input, and output tokens at the recorded list prices; reasoning tokens are included in output cost.

CS224W, Stanford University. Project milestone.

Changes from the proposal. We use 800 questions for these initial experiments instead of the proposed 1,800. We switched to GPT-5 because of endpoint throughput limits. We use five-fold cross-validation instead of a single train/test split and include historical LiveBench questions. Since these are public benchmarks, we cannot rule out training-data contamination. We report question-level averages here and plan to add dataset-level results.

Splits and reward. Each fold contains 640 training and 160 test questions. All nine responses to a question remain in the same fold. For configuration $a = (m, e)$, we use

$$r(q, a) = \alpha\widetilde{p}(q, a) - \beta\widetilde{c}(q, a), \qquad y_{q,a} = \mathbf{1}\Big[a = \arg\max_{a'} r(q, a')\Big]. \tag{1}$$

Here $\widetilde{p}$ is min–max normalized benchmark quality and $\widetilde{c}$ is min–max normalized log dollar cost. We compute the normalization bounds from the original training fold and use the same bounds for all methods. Ties are resolved by lower dollar cost, then a fixed configuration order. We train separate routers for quality-first $(\alpha, \beta) = (1, 0)$, balanced $(0.5, 0.5)$, and cost-first $(0.2, 0.8)$ objectives. Each question is tested once, using a router trained on the other folds.

3 Routing experiments

Frozen-encoder MLP. We encode query text, including answer choices, and task descriptions with `all-MiniLM-L6-v2`. The encoder uses at most 256 tokens and produces normalized 384-dimensional mean-pooled representations. Separate $384 \to 32 \to 32$ MLPs project query and task features. Learned model and effort embeddings each have dimension 32. Their concatenation is scored by a $128 \to 32 \to 1$ MLP for each of the nine configurations. This router has 31,105 trainable parameters and no graph message passing. We train with binary cross-entropy against the one-hot labels, Adam, learning rate 10^{-3}, batch size 32, and 1,000 epochs, retaining the final checkpoint.

Rebalanced training data. The cheapest configuration, nano low, accounts for 67.75%, 67.75%, and 77.66% of training labels across the three objectives. We create a separate *Balance* training subset for each fold and objective by removing questions whose best configuration is nano low. This reduces each training fold from 640 to 320 questions while keeping the same dataset ratio: 100 LiveBench, 120 SuperGPQA, and 100 MMLU-Pro questions. We also keep the task proportions within each dataset approximately the same. Nano-low label shares fall to 35.50%, 35.50%, and 55.31%, respectively. We keep the original gold labels, normalization bounds, and 160 test questions.

Joint encoder and head training. We then fine-tune both the encoder and routing head on the Balance subsets, using two initializations. *Warm start* initializes the routing head from the corresponding trained Balance MLP and then updates the encoder and head jointly. *Fresh head* initializes the head randomly and also updates both jointly; its encoder still starts from pretrained MiniLM. Unfreezing the encoder increases the trainable parameter count to 22,596,481, approximately $726\times$ the frozen-encoder baseline. We use AdamW with encoder learning rate 2×10^{-5}, head learning rate 10^{-4}, weight decay 0.01, gradient clipping at 1, and effective batch size 32 (microbatch 8, accumulation 4). Both variants train for 50 epochs, saving checkpoints every 10 epochs and evaluating the final checkpoint. As in the MLP experiments, we use the final checkpoint without a validation set.

Evaluation. We train the frozen-encoder MLP with five seeds. For full fine-tuning, we have completed seed 0 for all five folds and three settings. Table 2 uses seed 0 for all learned routers so that the comparison uses the same seed and 800 test questions. Warm start also includes the earlier MLP training. Five-seed MLP results are in the repository.

4 Results

Effect of rebalancing. Balance MLP improves quality-first reward from 0.6741 to 0.6923, but lowers balanced reward from 0.1835 to 0.1708 and cost-first reward from -0.0865 to -0.1266. Removing nano-low examples helps the quality-first setting, but hurts reward when cost is also considered.

Table 2: Mean held-out reward, seed 0, five folds; higher is better. Bold marks the best non-oracle result in each column. The oracle selects configurations using the scored responses.

Strategy	Quality-first	Balanced	Cost-first
GPT-5-nano low	0.6005	**0.1879**	**-0.0597**
GPT-5-mini low	0.7292	0.1816	-0.1470
GPT-5 high	**0.7942**	0.0606	-0.3795
Original MLP	0.6741	0.1835	-0.0865
Balance MLP	0.6923	0.1708	-0.1266
Full fine-tuning, warm start	0.6916	0.1801	-0.1088
Full fine-tuning, fresh head	0.6005	**0.1879**	**-0.0597**
Quality-first oracle	0.8975	0.3055	-0.0498

Effect of encoder fine-tuning. Warm-started fine-tuning gives almost the same quality-first result as Balance MLP (0.6916 versus 0.6923). It improves balanced and cost-first rewards to 0.1801 and -0.1088, but both remain below Original MLP. For the balanced objective, its mean quality rises from 0.6904 to 0.7088 while mean generation cost falls from \$0.002087 to \$0.001981 per query. However, its balanced reward is still below the original MLP result of 0.1835. When we initialize the head randomly, the router selects nano low for every test question under all three settings. It therefore gives the same results as always using nano low.

Comparison with fixed configurations. None of our learned routers beats the best fixed configuration for its setting among the nine candidates. GPT-5 high gives the highest quality, while nano low gives the highest reward in the two cost-aware settings. The quality-first oracle selects the highest-scoring response for each question, breaking ties by lowest cost; when no configuration earns full credit, it retains the best available score. It reaches quality 0.8975 at \$0.002346 per query. The oracle suggests room for better routing, but does not separately maximize each cost-aware reward.

5 Next steps

In these experiments, simply increasing the number of trainable parameters does not consistently improve routing performance. We have only run full fine-tuning with one seed so far, so we will check whether this result holds across more seeds. Our next step is to implement the proposed task–query–model–effort graph, building on GraphRouter [1]. We will compare it with the MLP on the same splits to test whether message passing helps beyond the shared input features. To prevent target leakage, we will mask all response edges for the query being predicted and exclude test outcomes from message passing. We also plan to remove effort nodes and replace task nodes with dataset-level nodes, as described in the proposal. We would appreciate feedback on the training objective, especially how to handle the large share of nano-low labels.

References

[1] Tao Feng, Yanzhen Shen, and Jiaxuan You. GraphRouter: A Graph-based Router for LLM Selections. ICLR, 2025. https://arxiv.org/abs/2410.03834.

[2] Yubo Wang et al. MMLU-Pro: A More Robust and Challenging Multi-Task Language Understanding Benchmark. NeurIPS Datasets and Benchmarks, 2024. https://arxiv.org/abs/2406.01574.

[3] SuperGPQA Team. SuperGPQA: Scaling LLM Evaluation across 285 Graduate Disciplines. 2025. https://arxiv.org/abs/2502.14739.

[4] Colin White et al. LiveBench: A Challenging, Contamination-Free LLM Benchmark. ICLR, 2025. https://arxiv.org/abs/2406.19314.